

Auteur : Nicolas Grelier, élève 1A

Encadrants : Vincent Gripon, Bastien Padeloup, Jean-Charles Vialatte

Destinataire : Amer Baghdadi



Rapport projet recherche

Rapport

Version 1 - 10/05/2016

Cursus Recherche

Année scolaire 2015-2016



RÉSUMÉ

La théorie des graphes est un outil très puissant en informatique. Cependant, les outils mathématiques ont souvent été développés dans le cas continu, et il est difficile de les appliquer à un graphe, qui contient dans la pratique un nombre fini de sommet. Notamment, il apparaît qu'en Deep Learning, il serait intéressant de posséder une définition de « translation » sur un graphe, par analogie à celle sur espace vectoriel. Il existe certains graphes sur lesquels on sait ce que devrait être une translation. Le but de notre travail a été de trouver une définition qui donne sur ces graphes-ci les translations souhaitées, mais qui soit suffisamment souple pour être adaptée aux autres graphes. Notre travail a consisté à trouver une définition de départ, puis à l'affiner petit à petit selon les résultats obtenus.

Sommaire

RÉSUMÉ.....	2
1. INTRODUCTION.....	4
2. GÉOMÉTRIE ET THÉORIE DES GRAPHS.....	4
2.1 INTÉRÊT DE LA TRANSLATION SUR GRAPHE.....	4
2.2 DIFFICULTÉS.....	4
3. CAS DES GRAPHS GRILLES CYCLIQUES.....	5
3.1 TRANSLATIONS ÉLÉMENTAIRES.....	5
3.2 AJOUT D'UN NOUVEAU CRITÈRE DANS LA DÉFINITION.....	6
3.3 TESTS INFORMATIQUES ET PREUVE.....	7
4. EXTENSION AUX AUTRES GRAPHS.....	7
4.1 PERTE D'INFORMATION.....	7
4.2 AMÉLIORATION DES CRITÈRES.....	9
4.3 AJOUT D'UN DERNIER CRITÈRE.....	9
5. CONCLUSION.....	10
RÉFÉRENCE BIBLIOGRAPHIQUE.....	11
ANNEXES.....	13
ANNEXE 1 -PREUVE.....	14
ANNEXE 2 -PROGRAMMES.....	16
1) Graphes grilles cycliques.....	16
2) Graphes grilles non cycliques première version.....	18
3) Graphes grilles non cycliques deuxième version.....	19

Index des illustrations

1. INTRODUCTION

Des chercheurs se sont déjà intéressés à la notion de translation sur graphe. Des travaux plus anciens en théorie spectrale des graphes ont fourni un outil de transformée de Fourier d'un graphe. Ainsi, la piste étudiée fut de considérer la transformée de Fourier d'un graphe pour en déduire ce que pourrait être une translation sur graphe. Cependant les résultats sont assez partiels. Visuellement, les « translations » trouvées ne correspondent pas du tout à ce qu'on voudrait. Notre travail a consisté à explorer une autre voie pour définir la translation sur graphe.

Dans un premier temps nous expliquerons pourquoi chercher une translation sur graphe, et pourquoi cela est difficile. Nos motivations sont d'une part liées au Deep Learning, mais aussi à des fondements mathématiques plus abstraits. Les graphes sur lesquels on connaît le mieux la notion de translation sont les graphes grilles. Nous sommes partis d'eux pour échafauder notre première définition. Cette définition doit donner sur les graphes grilles les translations, mais aussi pouvoir être adaptée aux autres graphes. En tentant de l'adapter, nous devons mettre à jour notre définition pour qu'elle convienne à tous les graphes étudiés.

2. GÉOMÉTRIE ET THÉORIE DES GRAPHES

2.1 INTÉRÊT DE LA TRANSLATION SUR GRAPHE

Dans le Deep learning, et notamment pour un réseau neuronal convolutif, l'outil principal est la convolution. On désire appliquer le même traitement de manière décalée aux différentes zones de récepteurs. Pour une image représentée sur une matrice de pixels, cela ne pose pas de problème. On décale d'un pixel verticalement ou horizontalement. On sait donc comment se déplacer sur l'image. On peut de même généraliser ce procédé sans peine aux dimensions supérieures. Cependant, dans le cas pratique, on est parfois amené à traiter des graphes avec des structures ne possédant pas de telles régularités. C'est pourquoi nous cherchons s'il existe une translation sur ce genre de graphes. Un graphe $G=(V,E)$ est défini par un ensemble de sommets V et un ensemble d'arêtes $E \subset V \times V$. On définit alors la distance d entre deux sommets comme la longueur du plus court chemin les reliant. Si le graphe est pondéré on compte alors la somme du poids des arêtes de ce chemin, sinon on compte un poids de 1 par arête. La translation en géométrie est un déplacement sans rotation, retournement ou déformation. Sur un graphe, nous cherchons donc une application $f:V \rightarrow V$, telle que pour tout couple de sommets (u,v) , on ait $d(u,f(u))=d(v,f(v))$ afin d'empêcher les rotations et les retournements, et $d(u,v)=d(f(u),f(v))$ pour empêcher les déformations.

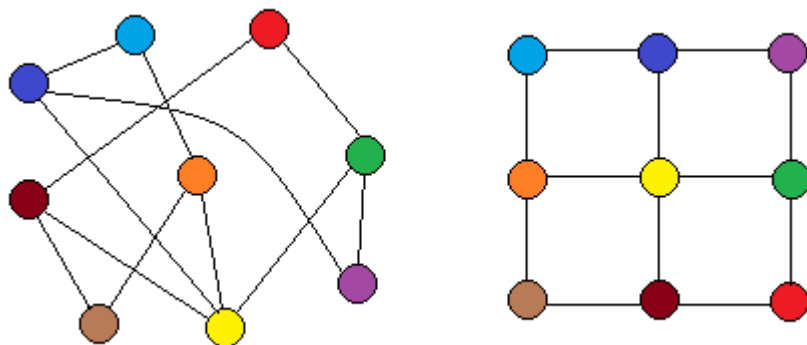
2.2 DIFFICULTÉS

Rien n'assure que les propriétés que nous cherchons puissent être vérifiées pour tout graphe. En fait, il existe une quantité certaine de contre-exemples. Sur le graphe ci-dessous, c'est par exemple impossible. Il nous faut donc trouver des conditions plus souples.



Sur un espace vectoriel, la translation selon le vecteur x , consiste à associer à tout vecteur u , le vecteur $u+x$. Dans toute la suite, on définira un graphe $G=(V,E)$ où V est l'ensemble des sommets, et E l'ensemble des arêtes. Sur un graphe, on ne peut plus appliquer un tel procédé. Premièrement dans le cas pratique, l'ensemble des sommets est fini. On ne peut donc pas assurer

l'existence de $u+x$ pour tout $u \in V$, par analogie aux espaces vectoriels. La deuxième grande difficulté est qu'un graphe n'est placé sur aucun support. Lorsqu'on le représente, le placement des sommets n'a aucune importance. Un graphe n'est qu'un ensemble de sommets et un ensemble d'arêtes qui les relient. Par exemple les deux graphes ci-dessous sont identiques.



Par ailleurs certains graphes sont pondérés, il faudrait alors potentiellement tenir compte du poids des arêtes. On peut ajouter à cela le fait que certains graphes soient orientés, ou que certaines arêtes bouclent sur un même sommet, mais ce ne sont pas des cas qui nous préoccupent en pratique.

Nous cherchons donc à définir une notion de translation, qui est originellement une notion de géométrie, sur des graphes. Il y a plusieurs propriétés que nous aimerions retrouver, mais rien ne nous garantit que cela est possible.

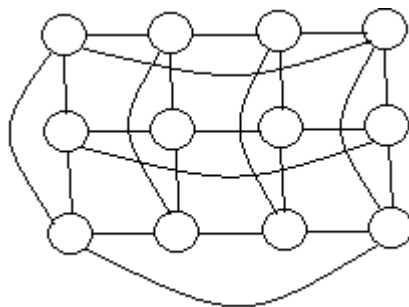
3. CAS DES GRAPHES GRILLES CYCLIQUES

3.1 TRANSLATIONS ÉLÉMENTAIRES

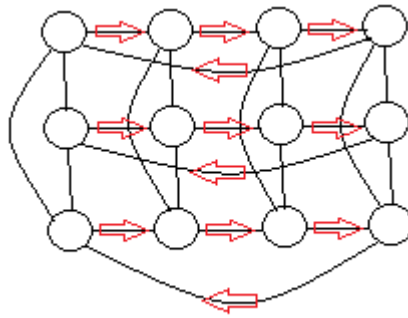
En géométrie, il n'existe pas de notion de translation élémentaire, car l'espace est continu. Cependant, en théorie des graphes, « l'espace » est fini. Nous pouvons donc chercher des translations élémentaires, les plus courtes possibles, les autres translations s'obtenant par composition de translations élémentaires. Nous pensons que cela simplifie la définition de translation que nous obtiendrons.

Nous avons vu dans la partie précédente les propriétés que nous désirons, ainsi que les difficultés que nous pouvons rencontrer. Il existe un type de graphe sur lequel on ne rencontre pas ces problèmes : les graphes grilles cycliques. Notre but dans cette partie est de trouver une définition de translation sur ces graphes, qui pourra être étendue aux autres graphes.

On définit un graphe grille cyclique de paramètres $(d, (k_1, k_2, \dots, k_d))$ comme étant un graphe semblable à une grille de dimension d où les $(k_1, k_2, \dots, k_d)$ sont les longueurs de chaque dimension. Les sommets aux extrémités sont reliés entre eux. Voir en annexe 1 pour une définition rigoureuse.



Ci-dessus le graphe grille cyclique de paramètres $(2(4,3))$.



La translation élémentaire sur un tel graphe correspond au fait d'envoyer chaque sommet sur un de ses voisins, suivant un même sens. On a représenté sur le graphe ci-dessus l'une des translations possibles, où chaque flèche part d'un sommet vers son image. Ainsi, on vérifie toutes les propriétés souhaitées que l'on a énoncé lors de la partie précédente. Il nous faut donc trouver une définition de translation qui permette de retrouver les applications que l'on désire, sans pour autant utiliser le fait que l'on se trouve sur un graphe grille cyclique, sinon on ne pourrait pas étendre cette définition aux autres graphes.

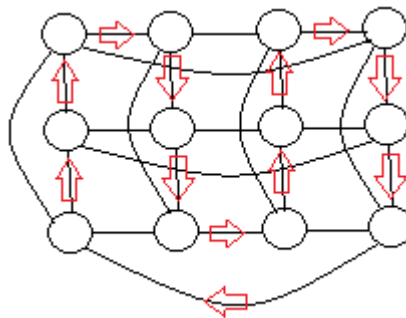
Il apparaît deux premiers critères :

- une translation est bijective,
- pour une translation élémentaire, l'image d'un sommet est l'un de ses voisins.

Ces deux critères sont dans notre première définition des conditions nécessaires pour être une translation.

3.2 AJOUT D'UN NOUVEAU CRITÈRE DANS LA DÉFINITION

Lorsque l'on cherche les applications qui vérifient ces deux critères, on trouve naturellement les translations, ainsi que d'autres applications qu'on voudrait éviter. Par l'application ci-dessous n'est clairement pas une translation.



Nous ajoutons dès lors un troisième critère : si deux sommets sont voisins, alors leurs images sont voisines. Ceci permet d'éviter le contre-exemple montré ci-dessus. Ce critère vient notamment du fait que l'on cherche à empêcher les déformations.

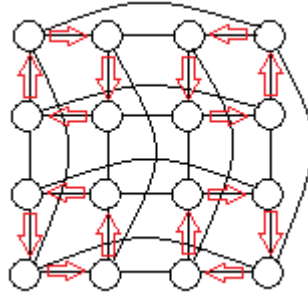
Rappelons la définition sur laquelle nous sommes arrivés, une application f est une translation sur un graphe $G=(V,E)$ si :

- f est bijective,
- $\forall v \in V, (v, f(v)) \in E$,

$$- \forall u, v \in V, (u, v) \in E \Rightarrow (f(u), f(v)) \in E .$$

3.3 TESTS INFORMATIQUES ET PREUVE

Fort de cette définition, nous avons implémenté un programme donnant toutes les applications vérifiant ces trois critères sur un graphe grille. Le code en OCaml est fourni en annexe. Pour des graphes assez petits on trouve des applications qui ne sont pas des translations.



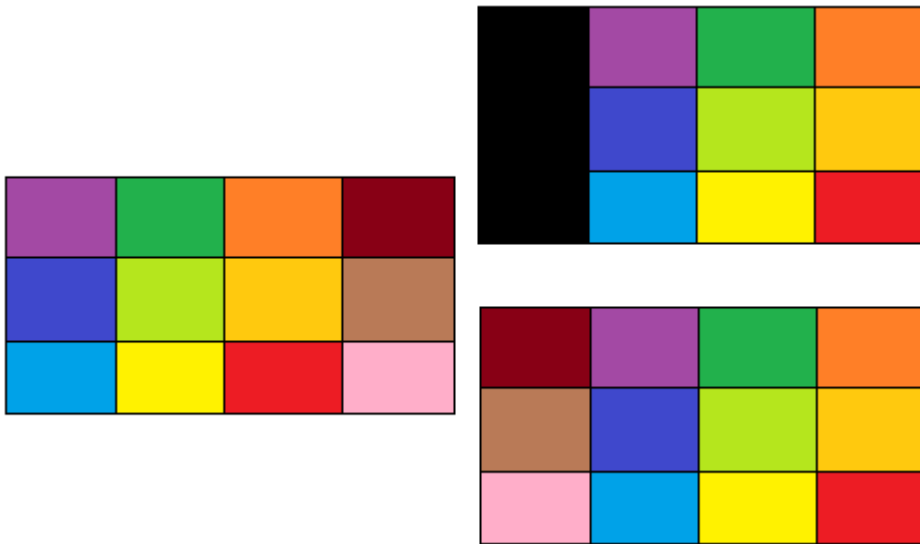
Cependant, pour un graphe dont les longueurs de chaque dimension sont supérieures à cinq, alors on ne retrouve plus que les translations souhaitées. Nous avons donc prouvé ce résultat. Une preuve complète figure en annexe 1. Nous en donnons cependant le principe général ici. On se donne une application f qui vérifie les trois critères. On considère ensuite un sommet v quelconque, ainsi que son image $f(v)$. Grâce aux trois critères que vérifie f , ainsi qu'au fait que le graphe est assez grand, on montre alors que tous les sommets voisins de v sont visuellement « envoyés dans le même sens ». Par ailleurs il existe un chemin entre tout couple de sommets du graphe. Puisque tous les sommets d'un chemin sont « envoyés dans le même sens », on en déduit que tous les sommets sont « envoyés dans le même sens », ce qui correspond à une translation.

Nous sommes effectivement parvenu à donner trois critères pouvant s'appliquer à tout graphe, qui donnent les translations dans le cas des graphes grilles cycliques assez grands. Cependant, il existe un autre type de graphe sur lequel on aimerait trouver des translations, et il nous faut dès lors nous assurer que notre définition fonctionne sur ces graphes.

4. EXTENSION AUX AUTRES GRAPHES

4.1 PERTE D'INFORMATION

Fort de notre définition, nous allons l'appliquer aux graphes grilles non cycliques, ce qui correspond entre autres aux images. Sur une image en deux dimensions, nous savons ce à quoi correspond la translation : c'est le fait d'envoyer l'information contenue par chaque pixel sur un des pixels d'à côté. La question est de savoir ce qu'il advient de l'information contenue par les pixels situés à l'extrémité. Il y a deux choix que l'on peut envisager : perdre l'information ou bien l'envoyer à l'autre extrémité. Ici à gauche une image, et à droite les deux manières envisagées d'exécuter une translation vers la droite.



Sur un graphe, le choix est le même. On peut choisir de perdre de l'information, ou envisager de transmettre de l'information alors qu'il n'y a pas d'arêtes pour relier les sommets. Nous avons estimé la première solution comme la meilleure a priori. En effet, rappelons que sur un graphe il n'existe pas de notion d'extrémité gauche ou droite par exemple. De plus, l'idée de conservation de l'information^[1] (ou conservation de l'énergie), étudiée par B.GIRAULT, P.GONCALVES et E.FLEURY, n'a pas donné les résultats escomptés. Nous amenons donc dans la définition la possibilité d'envoyer un sommet vers une sorte de « trou noir », noté ω , qui correspond à la destruction de l'information portée par le sommet. Il nous faut donc décrire quand envoyer un sommet vers le trou noir, sans pour autant s'appuyer sur la structure du graphe grille, car sinon on ne pourrait adapter la définition aux autres graphes.

Notre première idée fut de minimiser la perte d'information. On obtint alors la définition suivante, en notant $V_{0,f}$ l'ensemble des sommets qui ne sont pas envoyés vers ω par f .

Une application $f: V \rightarrow V \times \{\omega\}$ respecte la topologie du graphe $G = (V, E)$ si :

- 1) $f|_{V_0}$ est injective,
- 2) $\forall v \in V_0, (v, f(v)) \in E$,
- 3) $\forall u, v \in V_0, (u, v) \in E \Rightarrow (f(u), f(v)) \in E$.

On remarque qu'on ne peut pas exiger la bijectivité de $f|_{V_0}$, seule l'injectivité nous intéresse puisqu'il y a perte d'information.

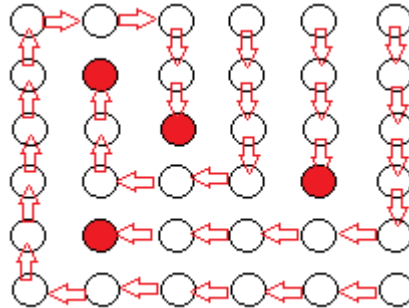
Une application f est une translation si :

- f respecte la topologie,
- $\forall g: V \rightarrow V \times \{\omega\}$ qui respecte la topologie, on a $|V_{0,f}| \geq |V_{0,g}|$.

C'est-à-dire que parmi toutes les applications qui préservent la topologie, on ne garde que celles qui minimisent la perte d'information. On remarque que cela ne modifie pas les résultats de la partie précédente.

4.2 AMÉLIORATION DES CRITÈRES

Avec un nouveau programme dont le code est donné en annexe, on voit que les applications obtenues ne sont pas les translations. Il y a d'autres applications que l'on désirerait empêcher, et ce pour toute taille de graphes. Ci-dessous un graphe grille non cyclique dont les arêtes n'ont pas été représentées pour plus de lisibilité. Les sommets en rouge sont ceux envoyés vers ω . Cette application minimise plus la perte d'information mais n'est pas une translation.

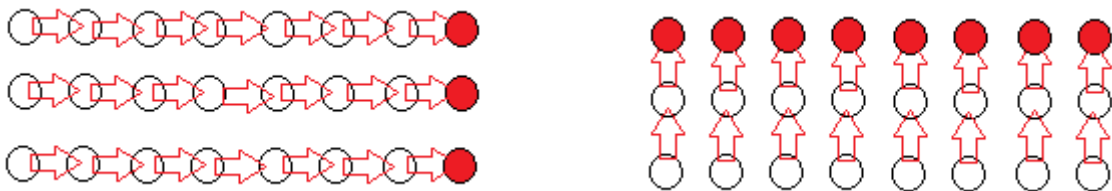


Nous nous sommes alors aperçus que si f est une translation, alors f^{-1} doit l'être aussi. Il est donc normal que si des sommets ont des images voisines, alors eux-mêmes doivent être voisins. C'est pourquoi nous avons modifié le troisième critère ainsi : . L'implication est devenue une équivalence. On remarque que l'application du schéma ci-dessus ne vérifie pas ce nouveau critère.

Grâce à un nouveau programme informatique, nous avons vérifié que pour des graphes grilles de dimension deux « carrés », il y a encore quelques applications non souhaitées pour les petits graphes, mais que pour un graphe de taille 6×6 , on ne trouve plus que les translations. De plus, les problèmes rencontrés viennent vraisemblablement des effets de bord. On peut donc conjecturer que plus le graphe est grand, plus notre définition fonctionnera.

4.3 AJOUT D'UN DERNIER CRITÈRE

Comme on vient de le voir, notre définition donne des résultats satisfaisants sur les graphes grilles « carrés ». Cependant, un problème survient lorsqu'on l'applique aux autres graphes grilles. Étudions par exemple le graphe grille de dimension deux de paramètres $(8,3)$.



Cette application envoie trois sommets sur le « trou noir ». Et celle-là en envoie huit.

Les deux sont des translations, mais seule la première répond à nos critères. C'est donc que notre définition n'est pas encore bonne. Pour pallier cette difficulté, nous allons devoir la modifier. Nous gardons la définition d'application qui préserve la topologie comme avant.

Une application $f: V \rightarrow V \times \{\omega\}$ respecte la topologie du graphe $G = (V, E)$ si :

- 1) $f|_{V_0}$ est bijective,
- 2) $\forall v \in V_0, (v, f(v)) \in E$,
- 3) $\forall u, v \in V_0, (u, v) \in E \Leftrightarrow (f(u), f(v)) \in E$.

Nous adaptons en revanche l'autre définition. Une application f est une translation si :

- f respecte la topologie,
- $\forall v \in V, \forall g: V \rightarrow V \times \{\omega\}$ qui respecte la topologie et telle que $g(x) = f(x)$, on a $|V_{0,f}| \geq |V_{0,g}|$.

De manière plus littérale, cela signifie qu'une application $f: V \rightarrow V \times \{\omega\}$ est une translation si elle respecte la topologie, et que pour tout sommet du graphe, s'il existe une autre application qui respecte elle aussi la topologie et qui envoie ce sommet dans le même sens que f , alors elle fait perdre plus d'information. Expliquons pourquoi ce choix de critère. Si l'on admet que sur un graphe grille rectangulaire les applications qui minimisent la perte d'information sont les translations dans le sens de la longueur, alors les applications qui n'envoient aucun sommet dans le même sens que celles-ci, envoient alors chaque sommet dans le sens de la largeur. On montre sans trop de peines que les seules applications qui respectent la topologie, minimisent la perte d'information, et qui envoient chaque sommet selon le sens de largeur, sont les translations dans le sens de la largeur.

L'objectif de travail que nous avons à court terme est donc de montrer que les applications qui respectent la topologie, et qui minimisent la perte d'information, sont les translations dans le sens de la longueur. Si nous y parvenons, nous aurons donc réussi à trouver une définition de translation sur graphe qui donne le résultat souhaité sur les graphes grilles (les seuls sur lesquels nous avons des a priori sur ce qu'est une translation), et qui peut être appliquée à tout type de graphe. En effet tout graphe possède au moins une translation. L'application qui envoie tous les sommets sur le trou noir préserve la topologie. Il existe donc une application qui préserve la topologie et qui minimise la perte d'information. Cette application est une translation d'après notre définition. Même si nous ne disposons pas encore de preuve, les résultats de nos programmes informatiques nous portent à croire en la véracité de la conjecture énoncée précédemment.

5. CONCLUSION

Nous avons trouvé une définition de translation sur graphe qui semble être satisfaisante. Elle donne le résultat souhaité sur la plupart des graphes grilles, et peut être adaptée à d'autres graphes.

Une lacune est qu'on ne sait pas évaluer à quel point un graphe admet des translations ou non. Par exemple un graphe sur lequel les translations envoient presque tous les sommets sur le trou noir n'est pas très intéressant. Il faudrait vérifier que pour tout graphe, les translations ne perdent pas trop d'information. Ou alors il faut accepter que certains graphes ne possèdent pas de translation. Une continuation serait de savoir si notre définition peut être adaptée aux graphes pondérés, ou encore de savoir si notre définition est assez résiliente : si on prend un graphe grille et qu'on le modifie légèrement (en ajoutant ou retirant une arête par exemple), les résultats restent-ils cohérents ?

Référence Bibliographique

- [1] BENJAMIN GIRAULT, PAULO GONCALVES, ERIC FLEURY. « Translation and Stationarity for Graph Signals ». [Research Report] RR-8719, Ecole Normale Supérieure de Lyon; Inria Rhône-Alpes; INRIA. 2015. <hal-01144991>

Annexes

ANNEXE 1 - PREUVE

Ceci est un extrait d'un article scientifique en anglais que nous, Nicolas Grelier, Vincent Gripon, Bastien Padeloup et Jean-Charles Vialatte, comptons publier par la suite.

II. NOTATIONS AND DEFINITIONS

Notation 1. Let us consider a graph $\mathcal{G} = \langle V, E \rangle$ where V is the set of vertices and $E \subset V \times V$ is the set of edges. Throughout this paper, we only consider graphs that are simple ($\forall v \in V, (v, v) \notin E$) and symmetric ($\forall v, v' \in V, (v, v') \in E \Rightarrow (v', v) \in E$). Thus, by abusing notations we make no distinction between edges (v, v') and (v', v) .

The graphs we want to consider in this section correspond typically to regular domains such as one dimension line graphs or two dimensions grids. More generally, let us consider an integer $d \in \mathbb{N}^*$, that is the dimension of the cyclic grid graph. We denote by $(k_i)_{1 \leq i \leq d}$ the length of each dimension and consider that $\forall i \in \llbracket 1, d \rrbracket, k_i \geq 5$.

We introduce the Cartesian product group $\mathcal{E} = \langle V \triangleq (\mathbb{Z}/k_1\mathbb{Z}) \times (\mathbb{Z}/k_2\mathbb{Z}) \times \dots \times (\mathbb{Z}/k_d\mathbb{Z}), + \rangle$. Elements in V are k -tuples of the form $\langle x_1, x_2, \dots, x_d \rangle$. We denote the canonical basis elements $(e^i)_{1 \leq i \leq d}$.

Two elements x, x' of V are said to be *neighbors* if all their coordinates are equal but the i -th one and $x_i - x'_i = \pm 1$.

Definition 1. The cyclic grid graph with parameters d and (k_i) is the graph $\mathcal{G} = \langle V, E \rangle$ such that $(x, x') \in E$ if and only if x and x' are neighbors.

Figure 1 depicts an example of a cyclic grid graph with parameters $d = 2$ and $k = (6, 5)$.

Definition 2. We say that an application f from V to V preserves the topology of $\mathcal{G}$ if :

1) f is bijective,

2) for all vertex x in V , $f(x)$ is a neighbor of x ,

3) for all couple $(x, x') \in V^2, (x, x') \in E \Leftrightarrow (f(x), f(x')) \in E$.

Definition 3. We say that an application f from V to V is an elementary graphical translation on $\mathcal{G}$ if there is v , with $v = \pm e^i$, such that $\forall x \in V, f(x) = x + v$.

Note that an elementary graphical translation is such that, visually, all vertices are pushed in the same direction.

III. RESULTS

In this section we aim at showing that for cyclic grid graphs, elementary graphical translations and applications which preserves the topology are identical.

We first show that topologically preserving applications are elementary graphical translations.

Lemma 1. Contamination lemma: *Let f be a function that preserves the topology of a cyclic grid graph $\mathcal{G} = \langle V, E \rangle$, with parameters d and (k_i) , and let x_0 be in V . Let $v = f(x_0) - x_0$, then $\forall x \in V$ neighbor of x_0 , $f(x) = x + v$. To begin with, we will consider that $x \neq f(x_0)$, for it is simpler in that case, before generalizing to every neighbors of x_0 .*

Proof: First we show that $f(x_0 - v) = x_0$. As a matter of fact, properties 2) and 3) of topologically preserving functions give us that $f(x_0 - v) = (x_0 - v) \pm v$. Moreover we have $\forall i \in [1, d], k_i \geq 5$, thus $f(x_0 - v) = x_0$. Let x be in $V - \{x_0 + v, x_0 - v\}$, such as x_0 and x are neighbors. Property 3) gives us that $(f(x), f(x_0))$ is in E . Moreover, Property 2) gives us that $(x, f(x))$ is in E , so $f(x) = x_0$ or $f(x) = x + v$. Furthermore $f(x) \neq x_0$ because f is a bijection and $f(x_0 - v) = x_0$. We conclude: $f(x) = x + v$.

Let $x = x_0 + v$, then we have $f(x) = x \pm v$ because of the precedent result applied to x . Furthermore, as proven above, we have $f(x_0 - v) = x_0$. Thus $f(x) = x + v$ because of the bijectivity of f . Else we use the first lemma. Thus $f(x) = x + v$. For all x neighbor of x_0 , $f(x) = x + v$. ■

Theorem 1. *Let f be that preserves the topology of a cyclic grid graph, then f is a graphical translation.*

Proof: A cyclic grid graph is a connected graph. Let $v = f(0) - 0 = f(0)$. We have $\exists i, v = \pm e^i$ because a vertex and its image are neighbors. Let x be in V . There is a path between 0 and x . Because of lemma 2, for all vertex u in this path, $f(u) = u + v$. Thus $f(x) = x + v$. ■

Conversely, one can show that elementary graphical translations preserves the topology of a cyclic grid graph. The proof is straightforward and therefore omitted:

Proposition 1. *Let f be a graphical translation on $\mathcal{G}$, then f respects the topology of $\mathcal{G}$.*

ANNEXE 2 - PROGRAMMES

Tous les programmes présentés sont en Ocaml.

1) Graphes grilles cycliques

```
(* ##### Test rules ##### *)

let ruleOne f =
  let t=Array.make (Array.length f) 0 in
  let boule = ref true and i= ref 0 in
  while !boule && !i<Array.length f do
    if t.(f.(!i))=1 then boule := false else t.(f.(!i))<-1;
    i:=!i+1;
  done;
  !boule;;

let rec mem elt liste = match liste with
  []->false
  |l::lst-> if elt=l then true else mem elt lst;;

let rec neighbor u v g = match g with
  []-> failwith("erreur")
  |l::lst->if u=0 then (mem v l)
  else neighbor (u-1) v lst;;

let ruleTwo f g =
  let boule = ref true and i = ref 0 in
  while !boule && (!i < Array.length f) do
    if not (neighbor !i f.(!i) g) then boule := false;
    i := !i +1;
  done;
  !boule;;

let rec testNeighbors u f listNeighbors g = match listNeighbors with
  []-> true
  |v::lst -> if not (neighbor f.(u) f.(v) g) then false else testNeighbors u f lst g;;

let ruleThree f g =
  let rec aux liste index = match liste with
    []->true
    |l::lst -> (testNeighbors index f l g)&&(aux lst (index+1))
  in
```



```

aux g 0;;

(* ##### Test all applications ##### *)

let rec testTwoThree liste g = match liste with (* If we test permutations there is no need to verify the first rule *)
| [] -> []
| f::lst -> let fNew = Array.of_list f in
if (ruleTwo fNew g)&&(ruleThree fNew g) then f:testTwoThree lst g) else testTwoThree lst g;;

let rec testOneThree liste g = match liste with
| [] -> []
| f::lst -> let fNew = Array.of_list f in
if (ruleOne fNew)&&(ruleThree fNew g) then f:testTwoThree lst g) else testTwoThree lst g;;

(* ##### Create applications ##### *)

(* Vertices Method *)
let rec interleave x lst = match lst with
| [] -> [[x]]
| hd::tl -> (x::lst) :: (List.map (fun y -> hd::y) (interleave x tl))

let rec permutations lst = match lst with
| hd::tl -> List.concat (List.map (interleave hd) (permutations tl))
| _ -> [lst];;

let verticesMethod g n =
let rec aux index=
if index = n-1 then [n-1] else index:aux (index+1))
in let liste = aux 0 in
testTwoThree (permutations liste) g;;

(* Edges Method *)
let rec listNeighbors u g = match g with
| [] -> failwith ("erreur")
| l::lst -> if u=0 then l else listNeighbors (u-1) lst;;

let addImage vertex liste g = (* [a,b,c] liste -> [[a::liste];[b::liste];[c::liste]] *)
let neighbors = listNeighbors vertex g in
let rec aux lst = match lst with
| [] -> []
| hd::tl -> (hd::liste):aux tl)
in aux neighbors;;

let edgesMethod g n =
let rec add index liste = match liste with
| [] -> if index = (n-1) then addImage index [] g else []
| l::lst -> (addImage index l g)@(add index lst)

```

```

in
let liste = ref [] in
for i= n-1 downto 0 do
    liste := add i !liste;
done;
testOneThree !liste g;;

```

2) Graphes grilles non cycliques première version

```

let rec mem elt liste = match liste with
|[]->false
|l::lst-> if elt=l then true else mem elt lst;;

```

(* Last Method *)

```

let rec listNeighbors u g = match g with
|[]-> failwith ("error, out of bounds")
|l::lst -> if u=0 then ((-1)::l) else listNeighbors (u-1) lst;; (* We add the Black hole *)

```

```

let verifyRuleThree vertex g image liste=
    let neighbors = listNeighbors vertex g in
    let rec aux previousVertices index = match previousVertices with (* index is the value of the first vertex
of previousVertices *)
    |[]->true
    |l::lst -> if l= -1 then aux lst (index+1) else begin
        let isNeighbor = mem index neighbors and imagesAreNeighbors = mem image (listNeighbors l g)
in
        ((not isNeighbor)||imagesAreNeighbors)&&(aux lst (index+1))
        end;
    in aux liste (vertex+1);;

```

```

let addImage vertex liste g = (* [a,b,c] liste -> [[a::liste];[b::liste];[c::liste]] *)
    let neighbors = listNeighbors vertex g in
    let rec aux lst = match lst with
    |[]->[]
    |hd::tl -> if hd= -1 then (hd::liste):aux tl) else begin
        let isBijective = not (mem hd liste) and imagesAreNeighbors = verifyRuleThree vertex g hd liste in
        if (isBijective && imagesAreNeighbors) then (hd::liste):aux tl) else aux tl
        end;
    in aux neighbors;;

```

```

(* ##### Cost ##### *)

let costFunction application min = (* number of vertices sent to the black hole *)
  let rec aux liste cost = match liste with
    |[]->cost
    |l::lst -> if cost>min then cost else if l = -1 then aux lst (cost+1) else aux lst cost;
  in aux application 0;;

let minimiseCost applicationsList n =
  let rec aux liste min chosenApplications = match liste with
    |[]-> chosenApplications
    |l::lst -> let cost = costFunction l min in
      if cost < min then aux lst cost []
      else if cost = min then aux lst cost (l::chosenApplications)
      else aux lst min chosenApplications;
  in aux applicationsList n [];;

(* ##### function ##### *)

let lastMethod g n =
  let rec add index liste = match liste with
    |[]-> if index = (n-1) then addImage index [] g else []
    |l::lst -> (addImage index l g)@(add index lst)
  in
  let liste = ref [] in
  for i= n-1 downto 0 do
    liste := add i !liste;
  done;
  let chosenList = minimiseCost !liste n in
  chosenList;;

```

3) Graphes grilles non cycliques deuxième version

Ici nous ne considérons que des graphes grilles non cycliques carrés.

```

let listNeighbors i long lst= (* if lst=[] we do not consider the black hole, else lst=[-1]*)
  let liste = ref lst in
  if i-long>= 0 then liste:= (i-long):!liste;
  if i+long < long*long then liste:= (i+long):!liste;
  if i mod long != 0 then liste:= (i-1):!liste;
  if i mod long != long-1 then liste:= (i+1):!liste;
  !liste;;

```

```

let isBijective image vect long =
  let neighbors= listNeighbors image long [] in
  let rec aux liste = match liste with
    |[]-> true
    |l::lst -> if vect.(l)=image then false else aux lst
  in aux neighbors;;

```

```

let firstWay vertex image vect long = (* if (x,y) is in E, then (f(x),f(y)) is in E *)
  let neighbors = listNeighbors vertex long [] and imageNeighbors = listNeighbors image long [-1] in
  let rec aux liste = match liste with
    |[]->true
    |l::lst -> if (vect.(l) != -2)&&(not (List.mem vect.(l) imageNeighbors)) then false else aux lst
  in aux neighbors;;

```

```

let secondWay vertex image vect long = (* if (f(x),f(y)) is in E, then (x,y) is in E *)
  let i= ref (vertex+1) and verify = ref true in
  let vertexNeighbors = listNeighbors vertex long [] and imageNeighbors = listNeighbors image long [] in
  while (!i<long*long)&& !verify do
    if (List.mem vect.(!i) imageNeighbors)&&(not (List.mem !i vertexNeighbors)) then verify:=false;
    i:= !i +1
  done;
  !verify;;

```

```

let addImage vertex couple n = (* [a,b,c] liste -> [[a::liste];[b::liste];[c::liste]] *)
  let vector = fst couple and cost = snd couple in
  let maximum = n in
  if cost > maximum then []
  else begin
    let neighbors = listNeighbors vertex n [-1] in
    let rec aux lst = match lst with
      |[]-> []
      |hd::tl -> if ((hd= -1)&&(cost != maximum)) then begin let vect = Array.copy vector in
        vect.(vertex)<- hd;
        (vect,cost+1):aux tl) end
      else if (hd = -1) then aux tl
      else if ((isBijective hd vector n) && (firstWay vertex hd vector n)&&(secondWay vertex hd
vector n)) then begin let vect = Array.copy vector in
        vect.(vertex)<- hd;
        (vect,cost):aux tl) end
      else aux tl
    in aux neighbors;
  end;;

```

```

let minCost liste n =
  let rec aux lst min minList = match lst with
    | [] -> minList
    | hd::tl -> let cost = snd hd in
      if cost < min then aux tl cost [hd]
      else if cost = min then aux tl cost (hd::minList)
      else aux tl min minList;
  in aux liste n [];;

let lastMethod n =
  let vect= Array.make (n*n) (-2) in
  vect.(n*n-1)<- n*n-2;
  let liste = ref [(vect,0)] in (* because of symmetry and assuming that the four coins can't be sent to the
black hole *)
  for i= n*n-2 downto 0 do
    let auxList = ref [] in
    let len = List.length (!liste) in
    for j=0 to (len-1) do
      auxList := (addImage i (List.hd !liste) n)@(!auxList);
      liste:= List.tl !liste;
    done;
    liste:= !auxList;
    (*liste := List.map (function addImage i x n )!liste;*)
  done;
  (*minCost (!liste) n;;*)
  !liste;;

```